

实验 5 Socket 网络程序设计

数据 231 235127 艾洋

实验目的：

理解 Socket 通信原理，掌握使用 Socket 和 ServerSocket 类进行 TCP Socket 通信的程序设计方法。

实验内容：

1、使用 ServerSocket 类和 Socket 类实现按如下协议通信的服务器端和客户端程序。

服务器程序的处理规则如下：

- 1) 向客户端程序发送“Hi, 你好, 欢迎连接到服务器上!请输入密码:”。
- 2) 若读口令次数超过 3 次, 则发送“密码错误次数多于 3 次, 退出登录!”给客户端, 程序退出。否则向下执行步骤 3)。
- 3) 读取客户端程序提供的口令。
- 4) 若口令不正确, 则发送“输入的密码不正确, 请重新输入!”给客户端, 并转步骤 2), 否则向下执行步骤 5)。
- 5) 发送“密码验证通过”给客户端程序。

客户端程序的处理规则如下：

- 1) 读取服务器反馈信息。
- 2) 若反馈信息不是“Hi, 你好, 欢迎连接到服务器上!请输入密码:”, 则提示“连接到了错误的服务器上”, 程序退出。否则向下执行步骤 3)。
- 3) 提示客户端用户输入密码并将输入的密码发送给服务器。
- 4) 读取服务器反馈信息。
- 5) 若反馈信息是“密码验证通过”, 则提示“用户验证通过”, 客户端可以和服务器进行一问一答式聊天。若反馈信息为“密码错误次数多于 3

次，退出登录！”，则程序退出；若反馈信息是“输入的密码不正确，请重新输入！”，则提示用户密码错误，重新输入，并转步骤 3）。

【设计思路】

服务器端设计思路：服务器通过ServerSocket监听指定端口，等待客户端连接。连接建立后，向客户端发送欢迎信息并提示输入密码，通过循环接收客户端输入的密码并验证，若密码错误次数超过3次则结束通信；若密码正确，进入聊天模式，不断接收客户端消息并转发回复，直到一方发送“bye”结束通信，最后关闭所有资源。

客户端设计思路：客户端通过Socket连接到服务器指定的IP和端口，接收服务器发送的欢迎信息后提示用户输入密码，将密码发送给服务器并接收验证结果，若密码错误根据提示重新输入，若错误次数超限则关闭连接；若密码正确，进入聊天模式，接收服务器消息并发送回复，直到一方发送“bye”结束通信，最后关闭所有资源。

【程序源代码】

一问一答

```
MyServer:
import java.net.*;
import java.io.*;
import java.util.*;
//一问一答
public class MyServer {
    public static void main(String args[]){
        try{
            ServerSocket ss = new ServerSocket(1234);//0-65535
            System.out.println("服务器已启动，等待客户端连接...");
            Socket s = ss.accept();//没人来就阻塞
            System.out.println("客户端已连接，IP 地址： " +
s.getInetAddress().getHostAddress());
            PrintStream out= new PrintStream(s.getOutputStream());
            BufferedReader in = new BufferedReader(new
InputStreamReader(s.getInputStream()));
            Scanner scan =new Scanner(System.in);

            out.println("Hi，你好，欢迎连接到服务器上!请输入密码：");
```

```

String sin,sout;
int count=0;//记录密码错误次数
while(count<3){
    sout=in.readLine();
    if(!sout.equals("10086")) {
        count++;
        out.println("输入的密码不正确，请重新输入！");
    }
    else{
        out.println("密码验证通过!");
        break;
    }
}

if(count>=3){
    out.println("密码错误次数多于 3 次，退出登录！");
    out.flush(); // 确保消息发出
    out.close();
    in.close();
    s.close();
    ss.close();
    return;
}
else{
    while(true){
        sin=in.readLine();
        System.out.println("客户端说: " + sin);
        sout=scan.nextLine();//这是读一行，next 只能读一个单词
        out.println(sout);
        if((sin.equals("bye") || sout.equals("bye"))){
            System.out.println("通信结束！");
            break;
        }
    }

    out.close();
    in.close();
    s.close();
    ss.close();
}

}
catch(IOException e){
    System.out.println(e.getMessage());
}

```

```
    }  
  }  
}
```

MyClient:

```
import java.net.*;  
import java.io.*;  
import java.util.*;  
//建立 Socket  
//打开输入输出流  
//通信  
//关闭输入输出流和 Socket  
  
//一问一答  
public class MyClient {  
    public static void main(String args[]){  
        try{  
            Socket s = new Socket("127.0.0.1",1234);//端口号一样才能连接上  
            PrintStream out= new PrintStream(s.getOutputStream());  
            BufferedReader in = new BufferedReader(new  
InputStreamReader(s.getInputStream()));//字节变字符再 Buffered  
Scanner scan =new Scanner(System.in);  
  
            String sin,sout;  
            sin=in.readLine();  
            if(!sin.equals("Hi, 你好, 欢迎连接到服务器上!请输入密码: ")){  
                out.close();  
                in.close();  
                s.close();  
            }  
            else{  
                System.out.println("请输入密码 ");  
                int attempt=0;  
                while (true) {  
                    sout = scan.nextLine();  
                    out.println(sout);  
                    sin = in.readLine();  
  
                    if (sin.equals("密码验证通过!")) {  
                        System.out.println("用户验证通过 ");  
  
                        System.out.print("请发送第一条消息: ");  
                        sout = scan.nextLine();
```

```

        out.println(sout);

        while (true) {
            // 然后等待服务器回复
            sin = in.readLine();
            if(sin == null || sin.equals("bye")) {
                System.out.println("通信结束! ");
                break;
            }

            System.out.println("服务器说: " + sin);
            System.out.print("请回复: ");
            sout = scan.nextLine();
            out.println(sout);

            if(sout.equals("bye")) {
                System.out.println("通信结束! ");
                break;
            }
        }

        out.close();
        in.close();
        s.close();
        break;
    }
    else if (sin.equals("密码错误次数多于 3 次, 退出登录! "))
    {
        System.out.println("密码错误次数多于 3 次, 退出登
        录! ");

        out.close();
        in.close();
        s.close();
        return;
    }
    else {

        System.out.println("输入的密码不正确, 请重新输入!
        ");

        attempt++;
        if(attempt >= 3) {
            System.out.println("密码错误次数超过 3 次, 连
            接已关闭");

            out.close();
        }
    }
}

```

```

    }
}
}
```

非一问一答

```
MyServerDuo
import java.io.*;
import java.net.*;

public class MyServerDuo {
    public static void main(String[] args) throws IOException {
        ServerSocket ss = new ServerSocket(1234);//0-65535
        System.out.println("服务器已启动，等待客户端连接...");
        Socket s = ss.accept();//没人来就阻塞
        System.out.println("客户端已连接，IP 地址： " +
            s.getInetAddress().getHostAddress());

        SReadThread srt=new SReadThread(s);
        SSendThread sst=new SSendThread(s);
        srt.start();
    }
}
```

MyServerDuo

```
// 接收消息线程
```

```

    }

    public void run() {
        try {
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println("客户端说: " + line);
            }
        }
        catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}

// 发送消息线程
class SSendThread extends Thread {
    private Socket socket;

    public SSendThread(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try {
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
            String line;
            while ((line = br.readLine()) != null) {
                out.println(line);
            }
        }
        catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

MyClientDuo

```

import java.io.*;
import java.net.*;

public class MyClientDuo {
    public static void main(String[] args) throws IOException {
        Socket s = new Socket("127.0.0.1",1234);//端口号一样才能连接上
        CReadThread crt =new CReadThread(s);
        CSendThread cst=new CSendThread(s);
        crt.start();
        cst.start();
    }
}

class CReadThread extends Thread {
    private Socket socket;

    public CReadThread(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try {
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println("服务器说: " + line);
            }
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}

class CSendThread extends Thread {
    private Socket socket;

    public CSendThread(Socket socket) {
        this.socket = socket;
    }

    public void run() {
        try {
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

```



```

        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        String line;
        while ((line = br.readLine()) != null) {
            out.println(line);
        }
    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
}
}
}

```

【试验结果】

```

(base) PS D:\autumn\CS_Experience\Java_Experience\Exp06> java MyClient
请输入密码
181881
输入的密码不正确，请重新输入！
18886
用户验证通过
请发送第一条消息：Hello!
服务器说：OK
请回复：I'fine!
服务器说：Mee too
请回复：bye
通信结束！
(base) PS D:\autumn\CS_Experience\Java_Experience\Exp06>

(base) PS D:\autumn\CS_Experience\Java_Experience\Exp06> java MyServer
服务器已启动，等待客户端连接...
客户端已连接，IP 地址：127.0.0.1
客户端说：Hello!
OK
客户端说：I'fine!
Mee too
客户端说：bye
bye
通信结束！
(base) PS D:\autumn\CS_Experience\Java_Experience\Exp06>

Anaconda PowerShell Prompt
(base) PS C:\Users\28385> cd D:\autumn\CS_Experience\Java_Experience\Exp06
(base) PS D:\autumn\CS_Experience\Java_Experience\Exp06> javac MyServerDuo.j
ava
(base) PS D:\autumn\CS_Experience\Java_Experience\Exp06> java MyServerDuo
服务器已启动，等待客户端连接...
客户端已连接，IP 地址：127.0.0.1
Hello!
Good morning!!
客户端说：Haha
客户端说：NiHao!!!
客户端说：8388388
客户端说：3993

Anaconda PowerShell Prompt
(base) PS C:\Users\28385> cd D:\autumn\CS_Experience\Java_Experience\Exp06
(base) PS D:\autumn\CS_Experience\Java_Experience\Exp06> javac MyClientDuo.j
ava
(base) PS D:\autumn\CS_Experience\Java_Experience\Exp06> java MyClientDuo
服务器说：Hello!
服务器说：Good morning!!
Haha
NiHao!!!
8388388
3993

```

【结果分析】

服务器端结果分析：在实验中，服务器成功启动并监听指定端口，能够接收客户端的连接请求。通过密码验证机制，有效限制了非法用户的访问，确保了通信的安全性。在聊天过程中，服务器能够实时接收客户端的消息并进行响应，实现了稳定的双向通信。当客户端发送“bye”时，服务器能够正确识别并结束通信，同时关闭所有相关资源，避免了资源泄露。整个服务器端的运行过程符合设计预期，展示了Socket通信在服务器端的应用效果。

客户端结果分析：客户端能够成功连接到服务器，并根据服务器的提示进行密码输入。在密码验证阶段，客户端能够正确处理密码错误的情况，提示用户重新输入，并在错误次数超过限制时关闭连接，保证了用户交互的友好性和逻辑性。进入聊天模式后，客户端能够实时接收服务器的消息并进行回复，实现了流畅的交互体验。当用户输入“bye”时，客户端能够正确结束通信并关闭资

源，确保了程序的稳定性和可靠性。通过实验，客户端成功实现了与服务器的交互功能，验证了Socket通信在客户端的应用可行性。

河北工业大学课程共享计划